Module 06 Lab - Regression and Classification Models

Objective: To understand the difference between regression and classification and to build your first linear models for both tasks.

In this lab, you will write the code to train and evaluate the models.

Part 1: Regression vs. Classification

This is the most fundamental distinction between types of supervised learning problems.

Regression: The goal is to predict a **continuous numerical value**.

Examples: Predicting the price of a house, the temperature tomorrow, or the stock price.

Classification: The goal is to predict a **discrete category or class label**.

Examples: Predicting if an email is spam or not spam, if a flower is a setosa, versicolor, or virginica, or if a customer will churn or not.

In this lab, we will tackle one of each.

Part 2: Linear Regression

Concept: Linear Regression is used to predict a continuous value. It works by finding the best-fitting straight line through the data points. The model learns a "slope" (coefficient) for each feature and an "intercept".

Problem: We will predict the `Fare` of a Titanic passenger based on their `Age` and `Pclass`.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
# Load and prepare the data
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')
# For simplicity, we'll drop rows with missing age
df.dropna(subset=['Age'], inplace=True)
# Define features and target
features = ['Age', 'Pclass']
target_reg = 'Fare'
X_reg = df[features]
y_reg = df[target_reg]
# Split the data
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)
```

Task 1: Train and Evaluate a Linear Regression Model

Your Task:

1. Create an instance of the `LinearRegression` model.
2. Train the model using the training data (`X_train_reg`, `y_train_reg`).
3. Make predictions on the test data.
4. Evaluate the model using `mean_squared_error`. This metric tells us the average of the squared differences between the predicted and actual values.

```
# --- ENTER YOUR CODE HERE ---
# 1. Create the model instance
lr_model = LinearRegression()
# X_train_scaled = scaler.fit_transform(X_train)
# X_test_scaled = scaler.transform(X_test)
# 2. Train the model
lr_model.fit(X_train_reg, y_train_reg)
# 3. Make predictions
y_pred_reg = lr_model.predict(X_test_reg)
# y_prob = model.predict_proda(X_test_scaled)
# 4. Evaluate the model
mse = mean_squared_error(y_test_reg, y_pred_reg)
```

```
print(f"Mean Squared Error for Fare Prediction: {mse:.2f}")
print(f"The square root of this is {np.sqrt(mse):.2f}, meaning our model is off by about $48 on average.")
print(lr_model.coef_)
```

```
Mean Squared Error for Fare Prediction: 3364.92
The square root of this is 58.01, meaning our model is off by about $48 on average.
[ -0.43695889 -38.29728935]
```

Part 3: Logistic Regression

Concept: Despite its name, Logistic Regression is a **classification** algorithm. It works by calculating the probability that a given input belongs to a certain class. It's one of the most widely used and interpretable classification models.

Problem: We will predict whether a passenger (`Survived`) based on their (`Age`), (`Pclass`), and (`Sex`).

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
# Prepare data for classification
# We need to encode 'Sex' column
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
# Define features and target
features_cls = ['Age', 'Pclass', 'Sex']
target_cls = 'Survived'
X_cls = df[features_cls]
y_cls = df[target_cls]
# Split the data
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(X_cls, y_cls, test_size=0.2, random_state=42)
```

Task 2: Train and Evaluate a Logistic Regression Model

Your Task:1. Create an instance of the (`LogisticRegression`) model.2. Train the model using the classification training data.3. Make predictions on the test data.4. Evaluate the model using (`accuracy_score`).

```
# --- ENTER YOUR CODE HERE ---
# 1. Create the model instance
log_model = LogisticRegression(solver='lbfgs', C=1.0, random_state=42)
# X_train_scaled = scaler.fit_transform(X_train)
# X_test_scaled = scaler.transform(X_test)
# 2. Train the model
log_model.fit(X_train_cls, y_train_cls)
# 3. Make predictions
y_pred_cls = log_model.predict(X_test_cls)
# 4. Evaluate the model
accuracy = accuracy_score(y_test_cls, y_pred_cls)
print(f"Accuracy for Survival Prediction: {accuracy:.2%}")
```

```
Accuracy for Survival Prediction: 74.83%
```

📝 Knowledge Check

Instructions: Answer the following questions in this markdown cell.

1. **In your own words, what is the key difference between a regression problem and a classification problem?**

Answer: I think that the main differance is grouping. There are not sepperate groups in problems we choose to solve with regression. Rather, there are levels and our model predicts what lever the data is on. Whereas, classification is where we you a model to determin the group of the data rather then the level of the data.

2. **The (`LinearRegression`) model has an attribute called (`.coef_`). After you train the model, print (`lr_model.coef_`). What do these numbers represent?**

Answer: It is the raio of change for the features. Where if a feature goes of or down the prodicted value goes up or down by that much.

3. **Why did we use (`mean_squared_error`) to evaluate the regression model but (`accuracy_score`) for the classification model?

Answer: We use different metrics because the two models solve completely different types of problems:

1. Linear Regression (mean_squared_error): Predicts a continuous, infinite numerical value, in this case a price. Because the model will rarely guess the exact penny correctly, we can't use an "all-or-nothing" accuracy score. Instead, we use MSE to measure how far away our numerical guesses were from the actual values on average.

2. Classification (accuracy_score): Predicts a discrete category or label,in this case an 'is' or 'is not'. Because a category is either 100% right or 100% wrong, we use accuracy to count the overall percentage of correct guesses the model made.
3. ** Why wouldn't accuracy be a good metric for the fare prediction task?

Answer: Because it is not a Yes or No type of answer.